Passing the security audit

Constraint Solving [B-KUL-H02A3A]: Project 1

Academic year 2026–2027

1 Introduction

A security team is testing your skills as a system administrator. You are asked to set up a shared filesystem that has to follow a strict security policy: files have owners, permissions, sub-directories, protections, and so on. The different requirements impact and constrain each other. Having followed the Constraint Solving course, you recognize that this is a configuration problem and decide to use CPMpy to search for a file layout and permissions that satisfy all of the requirements. Those requirements are split into seven stages, and each stage adds its constraints to a CPMpy model.

The use of LLMs and other AI tools (for example ChatGPT, Copilot, or an AI assistant in your IDE) is not allowed for this project. This is a project to flex your programming and modeling skills; it is a challenge for you, which is relatively easy to solve for people and machines with more experience. We tested PyCharm 2026's default code completion (which uses a local LLM) and that one is acceptable. If you use another IDE that has AI code completion enabled, please disable it or ask us if it is allowed. Together with your project, you will need to submit a filled-in GenAI Code of Conduct.

For the precise deadline, grading weight and academic integrity rules, please see the Toledo submission instructions.

2 Getting started

2.1 Files

- `project1_description.pdf`: this document.
- `project1.py`: the file you complete. It is the only code file you submit.
- `helpers.py`: functions that `project1.py` uses to load the instances and to print a solution. You do not need to read or change it.
- `instances.json`: problem instances (see section 3).
- `requirements.txt`: the Python packages you need.

2.2 Setup

We recommend PyCharm and Python 3.10 or newer.

1. Unzip the archive and open the folder `project1` in PyCharm (File > Open).
2. Set the project's Python interpreter to a new virtual environment with Python 3.10 or newer (Settings > Project > Python Interpreter > Add Interpreter).

3. In PyCharm's Terminal, run `pip install -r requirements.txt`.

4. Run `project1.py`. For every instance in `instances.json`, it solves your model and runs your checks.

2.3 Functions to implement

The assignment, described below, consists of seven stages. For each stage, complete the corresponding function `stage1`, ..., `stage7` in `project1.py`: it receives a CPMpy model, the decision variables created by `make_variables`, and the instance data. You should add the constraints of that stage to the model, and return the model. Note that you are allowed to create additional auxiliary decision variables as needed.

2.4 Testing your own model

The security team has its own tests, that they will use to check if you pass each of the stages. However, they are not sharing them with you, as that can give away part of the solution.

They do encourage you to write your own tests. It is optional, but it allows you to verify that the generated solutions allow or disallow what you intend. How else will you know that you will pass the audit?

A minimal checking framework is provided that you can extend: running `project1.py` solves your model on every instance in `instances.json` and runs your checks by calling the function `my_checker(data, variables)`. A failing assert stops the run.

The code at the bottom of `project1.py` is yours to change: remove stages from the tuple to look at fewer stages, or loop over fewer instances (for example `instances[:1]`).

3 Assignment

3.1 Permissions and owners

A data instance contains the following data:

- `n_files`: the number of files
- `n_users`: the number of users
- `files_owned`: for each user, the number of files they own (see stage 5)
- `protected`: the protected files (see stage 6)
- `id`: a label for the instance, used in the check messages; your model does not use it

All problem instances in `instances.json` follow this format.

There are three permission types, `0 = read`, `1 = write`, `2 = execute`.

For each file, we need to decide who owns it and which permissions it has. Files have consecutive numeric IDs, from 0 to `n_files - 1`.

Users also have consecutive numeric user IDs, where user `0` is the admin user and the other users have IDs from 1 to `n_users - 1`. Hence, we can represent ownership with an array of integer decision variables, one for every file, where the value is the user ID of the owner.

In the shared filesystem we have to configure, each file has separate permission flags for each user. More specifically, for each file ID `f`, user ID `u`, and permission type `p`, we need to decide whether user `u` has permission `p` for file `f`, or not.

The decision variables are pre-created in `make_variables(data)`. Use these decision variables to model the following 7 policy stages:

Stage 1 For compatibility reasons, the file with ID `0` needs to be owned by the admin. The admin also needs to be able to repair any file, so that account needs `write` permission on every file.

Stage 2 Every user needs `read` and `write` permission on the files they own, so that an owner can open and change their own file. A user who changes a file needs to be able to see it, so `write` permission needs to come with `read` permission on that same file. Furthermore, a user who executes a file also needs to be able to change it, so `execute` permission needs `write` permission on that same file.

3.2 Directories

A directory is a file that contains other files, and a file that contains no files is not a directory. Every file, directory or not, has an owner and permissions, and the directory structure influences the permissions. The files can be seen as nodes in a directed graph, with an arc (a directed edge) from each directory to each file it contains. To keep track of which files contain which, we introduce Boolean decision variable `arcs[i, j]` for each file `i` and `j` which is true iff file `i` contains file `j`.

Now model the following stages:

Stage 3 A file cannot be stored in more than one directory, so every file has at most one parent directory. Moreover, there is exactly one *root* file, which is not stored in any directory.

Stage 4 All files need to be part of a single directory structure: starting from the root and going down through the directories, every file can be reached. Hint: one way to model this is to ensure there is an arc, in either direction, between any non-empty proper subset of the files and the remaining files (`itertools.combinations` from the Python standard library can be used to generate subsets).

Stage 5 A file inherits the active permissions of the directory that contains it: a user who has permissions granted on a directory also has those permissions on every file in that directory. Directories cannot be executed, by anyone. Finally, the disk allocation to every user is restricted: user `u` has to own exactly `files_owned[u]` files. In the first instance of `instances.json`, for example, the first two users own one file each and the last two users own two files each. Hint: you can use a global constraint for this last requirement.

3.3 Protected files and objective

The files in `protected` (files 3 and 4 in the first instance of `instances.json`) are *protected* files with special access rules. Model the following specification:

Stage 6 All protected files need to be stored together, in the same directory. A protected file cannot be a directory itself.

Stage 7 For auditing purposes, the directory of the protected files must contain at least one unprotected "log" file, and can contain other files too. An unprotected log file is not a directory, the admin needs to be able to read it, and every user who owns one or more protected files needs to be able to write to it. Furthermore, the protected files cannot be stored directly in the root directory, and each protected file must be owned by a different user.

Bonus stage (ungraded) To maximize security (and inconvenience), the number of active permissions should be as small as possible. Complete the function `objective` in `project1.py`. The loop at the bottom of `project1.py` will ask the solver to minimize `objective(variables, data)` when that value is not `None`.

4 Grading

We grade each stage separately. To grade stage k, we add our own constraints for stages 1 to $k-1$ to your constraints for stage k, so your stage k can rely on the requirements of the earlier stages (for example, stage 4 may use that there is exactly one root). We then check that your constraints for stage k are

- *not under-constrained*: together with our constraints for the earlier stages, they allow no solution that breaks a requirement of stage k;
- *not over-constrained*: every solution that satisfies all seven stages must also be accepted by your constraints for stage k.

Each of the seven stages counts equally; the bonus stage is not graded.

We grade on the instances in `instances.json` and on hidden instances in the same format. You are free to create your own instances to test your model.

5 Submission

Submit only `project1.py` and the filled-in GenAI Code of Conduct, through the Toledo assignment and before the deadline stated there (see Toledo for details). Do not submit `helpers.py` or the other files. Before you submit, check that

- running `project1.py` gives no errors;
- `make_variables` and the functions `stage1` to `stage7` keep their names and parameters;
- your file imports nothing besides `helpers`, the packages in `requirements.txt`, and the Python standard library.